

Resumo da Nova Seção — Explicações Didáticas e Ajustes

Marco de início desta seção: “Aqui começamos uma nova seção de interação. Quando eu pedir onde começamos a alteração atual, você identificará esse ponto.”

1) O que foi criado no repositório (repo simples, Nest no root)

- **src/main.ts** — bootstrap do Nest; ajuste para `void bootstrap()` devido à regra `no-floating-promises`.
 - **src/app.module.ts / app.controller.ts / app.service.ts** — módulo/rotas/serviço básicos do Nest.
 - **test/app.e2e-spec.ts** — teste E2E do `GET /`; correção para `import request from 'supertest'` + `await` (eliminando `no-unsafe-*`).
 - **package.json** — scripts: `start`, `start:dev`, `build`, `lint`, `test`, `test:e2e`.
 - **package-lock.json** — necessário para `npm ci` na CI.
 - **nest-cli.json** — configura paths/entry para o Nest CLI.
 - **tsconfig.json / tsconfig.build.json** — configs do TypeScript para dev e build.
 - **.eslintrc. / .prettierrc** — lint (qualidade) e prettier (formatação).
 - **.github/workflows/ci.yml** — pipeline com 3 jobs; adaptada para repo simples e preparada para detectar workspaces no futuro.
-

2) Conceitos explicados ao professor

2.1 Lint — `no-floating-promises`

- **O que é:** proíbe Promises criadas e não tratadas.
- **Por quê:** rejeições silenciosas causam bugs difíceis de rastrear.
- **Como resolver:** `await`, `.catch(...)` ou `void`.
- **Aplicação:** `void bootstrap();` em `main.ts`.

2.2 Lint — família `no-unsafe-*`

- **no-unsafe-call/member-access/return** evitam usar `any/unknown` sem refino.
- **Correção feita:** usar `import request from 'supertest'` e `await` no E2E, removendo retornos “unsafe”.

2.3 Teste E2E (End-to-End)

- **Conceito:** testa a aplicação como “caixa-preta” via HTTP, da entrada (controller) ao núcleo (services/DB) e de volta.
- **Diferença para unit/integration:** E2E cobre o caminho completo; unit/integration isolam partes.
- **No Nest:** `INestApplication` + `supertest` sobre `app.getHttpServer()`.

2.4 Jest configs

- **jest.config.ts**: config global (transform TS, descoberta de testes, cobertura, mapeamentos).
- **test/jest-e2e.json**: config específica para E2E (matches, rootDir, setup dedicado se necessário).

2.5 Scripts do package.json

- **start** (prod), **start:dev** (watch), **build** (TS → JS), **lint** (ESLint), **test** (unit), **test:e2e** (E2E).

2.6 Configs do Nest/TS

- **nest-cli.json**: instruções do Nest CLI (sourceRoot, entryFile, schematics, assets).
- **tsconfig.json** vs **tsconfig.build.json**: base para dev vs ajustes para build (exclusões, emit configs).
- **.eslintrc** e **.prettierrc**: qualidade vs formatação.

3) Ajuste de porta para 3001

Opção recomendada (via env)

```
// src/main.ts
const app = await NestFactory.create(AppModule);
const port = parseInt(process.env.PORT ?? '3001', 10);
await app.listen(port);
```

Executar localmente:

```
PORT=3001 npm run start:dev
```

Opção direta (fixa)

```
await app.listen(3001);
```

Observação: os testes E2E não dependem da porta porque usam `app.getHttpServer()`.

4) Como testar (resumo)

```
npm ci
npm run lint
npm run build
npm test
npm run test:e2e
```

```
PORT=3001 npm run start:dev  
# Acesse http://localhost:3001/
```

Se a CI falhar: - Verifique se o `package-lock.json` está versionado (para `npm ci`). - Confirme que o workflow está usando `npm run ...` (sem `-ws`) no repo simples. - Use `npm run lint -- --fix` para aplicar correções sugeridas pelo ESLint.

Este documento resume todos os diálogos e decisões a partir do marco da nova seção, servindo como base de aula e de auditoria do que foi feito.